What is Delta Live Tables in Databricks?

SWIPE

In this document, lets explore an important feature which is part of Databricks Premium: Delta Live Tables (DLT). In order to understand Delta Live Table , it is very important to understand what is an Autoloader and Spark Structured Streaming.

Delta Live Tables (DLT) is a framework designed to simplify the development of ETL pipelines using a declarative approach, addressing the challenges of traditional ETL processes. DLT streamlines orchestration, monitoring, batch and streaming data handling, data quality management, and more, making a data engineer's job significantly easier.

As mentioned earlier, DLT enables pipeline creation using a declarative approach. This means we focus on defining the desired outcome rather than specifying the exact steps to achieve it. The system automatically determines the necessary operations, reducing the complexity of managing individual instructions. In this article, I will demonstrate how this works in practice. Since it is a premium feature of Databricks , the demonstration which I will be showing here will not be on Standard Databricks Notebook.

Building the DLT pipelines involves 2 steps :

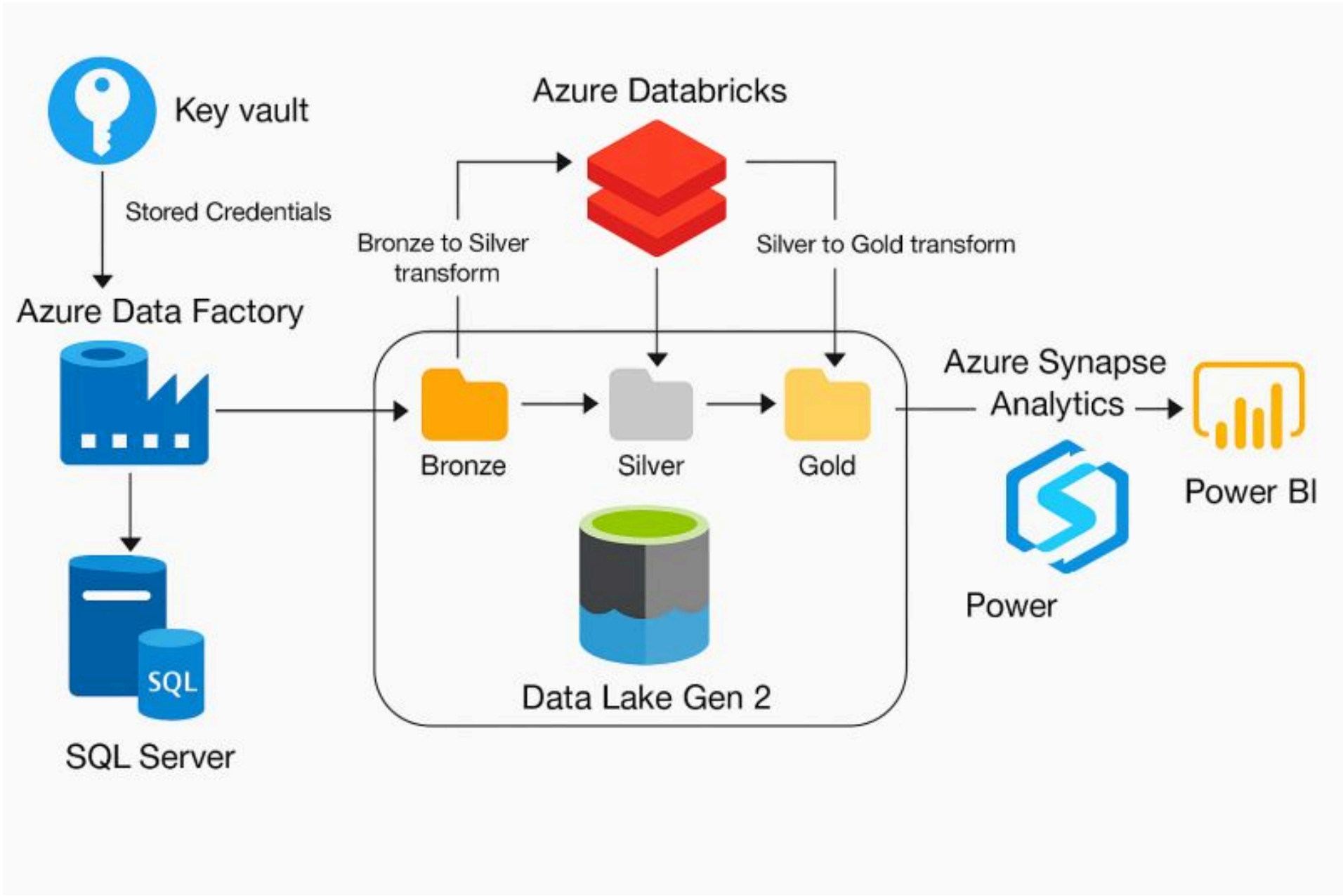
1. Declarative Code in SQL / Python

2. Configuring the Pipeline which runs the DLT Code.

Now, let's explore how Delta Live Tables (DLT) simplifies the implementation of Slowly Changing Dimension (SCD) Type 2 using the Medallion Architecture.

For those unfamiliar with the Medallion Architecture, it consists of four layers:

- Landing Layer: This is where raw files are ingested.
- Bronze Layer: Here, data engineers convert file formats and standardize column names.
- Silver Layer: This stage involves data quality checks and transformations, making the data suitable for consumption by data scientists for model training.
- Gold Layer: This is where business users and analysts perform queries to derive insights and support decision-making.


Key vault
Azure Databricks
Stored Credentials
Bronze to Silver transform
Silver to Gold transform
Azure Data Factory
Bronze
Silver
Gold
Azure Synapse Analytics
Power BI
Power
SQL
SQL Server
Data Lake Gen 2

Landing Layer : Initially, I mounted my Azure Storage to Databricks using an access key and mapped it to the DBFS location /mnt/landing.

```python
#%python
from pyspark.sql.functions import *
mountPoint = '/mnt/landing'
if not any(mount.mountPoint == mountPoint for mount in dbutils.fs.mounts()) :
        try :
                dbutils.fs.mount(
                        source = 'wasbs://landing@autoloadersareading.blob.core.windows.net'
                        mount_point = '/mnt/landing' ,
                        extra_configs = {               "fs.azure.account.key.autoloadersareading.blob.core.windows.net"
                       }
                )
                print("Mounted Successfully")
        except Exception as e :
                print("Failed to Mount" , e)
```

Bronze Layer : Since we are working with streaming data, the ideal approach is to use Autoloader for reading incoming data efficiently. In my Bronze layer, I created two streaming tables, orders_bronze and customer_bronze, using Autoloader:

```sql
CREATE STREAMING TABLE orders_bronze
AS
SELECT *,
       input_file_name() AS filename,
       current_timestamp() AS load_time
FROM cloud_files(
    '/mnt/landing/orders',
    'csv',
    map(
        "cloudFiles.inferColumnTypes", "true",
        "cloudFiles.schemaLocation", "/mnt/schema/orders"
    )
);
```

```sql
CREATE STREAMING TABLE customer_bronze
AS
SELECT *,
       input_file_name() AS filename,
       current_timestamp() AS load_time
FROM cloud_files(
    '/mnt/landing/customers',
    'csv',
    map(
        "cloudFiles.inferColumnTypes", "true",
        "cloudFiles.schemaLocation", "/mnt/schema/customers"
    )
);
```

A key aspect to note here is the use of Streaming Tables. These are essentially Delta Tables with built-in support for real-time data ingestion, allowing continuous data processing while avoiding unnecessary recomputation. It is one of core abstractions of DLT.

Silver Layer: Transforming and Implementing SCD Type 2

In the Silver Layer, I have divided the processing into two parts:

1. Data Cleaning: Extracting and transforming data from the Bronze layer. 2. Handling Updates: Implementing SCD Type 2 for the customer table while directly updating the orders table.

The key reason for applying SCD Type 2 on customer data is that tracking historical changes in customer attributes (e.g., city changes) provides valuable business insights, such as analyzing order behavior before and after a customer moves from City A to City B.

However, for orders, keeping historical versions of the same order is unnecessary, so a simple update operation is sufficient.

1. Data Cleaning:

--Cleaning of Data

```
CREATE STREAMING TABLE orders_silver_cleaned (
  CONSTRAINT valid_orders
    EXPECT (order_id IS NOT NULL)
    ON VIOLATION DROP ROW,

  CONSTRAINT valid_customer
    EXPECT (customer_id IS NOT NULL)
    ON VIOLATION DROP ROW
)
AS
SELECT *
FROM orders_bronze;
```

```sql
SELECT
    order_id,
    order_date,
    customer_id,
    total_amount,
    order_status,
    filename
FROM STREAM(live.orders_bronze);
```

```sql
CREATE STREAMING TABLE customer_silver_cleaned (
  CONSTRAINT valid_customer
    EXPECT (customer_id IS NOT NULL)
    ON VIOLATION DROP ROW
)
AS
SELECT
    CustomerID   AS customer_id,
    CustomerName AS customer_name,
    ContactNumber AS contact_number
FROM STREAM(live.customer_bronze);
```

Why we need to use LIVE and STREAM in the above code ?

LIVE keyword: Refers to tables created within the same DLT pipeline, ensuring consistency in table references.

STREAM function: Ensures the incremental processing of streaming data, preventing full table scans and optimizing performance.

2. Handling Updates:

For Customer :

```
CREATE STREAMING TABLE customers_silver;

APPLY CHANGES INTO live.customers_silver
FROM STREAM(live.customer_silver_cleaned)
KEYS (customer_id)
SEQUENCE BY load_time
STORED AS SCD TYPE 2;
```

For Orders :

```
CREATE STREAMING TABLE orders_silver;

APPLY CHANGES INTO live.orders_silver
FROM STREAM(live.orders_silver_cleaned)
KEYS (order_id)
SEQUENCE BY load_time;
```

To handle customer and order updates efficiently in the Silver Layer, I used APPLY CHANGES INTO, which enables incremental processing and ensures that data changes are properly managed.

In the above code , Keys : Defines the primary key to identify records for updates or inserts. Sequence By : Sorts data by a timestamp column to ensure the latest entry is processed correctly. Stored as SCD Type 2 (only for customers) keeps historical changes, while orders are simply updated with the latest status.

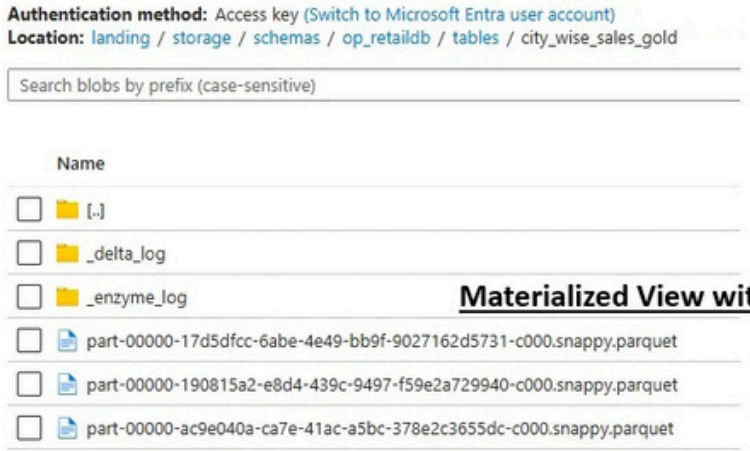
At first glance, it may seem like a lot of code just to implement SCD Type 2, but the key takeaway is that DLT focuses on what needs to be done rather than how it is done, eliminating the need to manually manage complex update logic.

Gold Layer : Take the data from silver layer and perform aggregation and load to gold layer.

```sql
CREATE MATERIALIZED VIEW city_wise_sales_gold AS
SELECT
    c.city,
    SUM(o.total_amount) AS total_sales
FROM live.orders_silver o
JOIN live.customers_silver c
  ON o.customer_id = c.customer_id
GROUP BY c.city;
```

Just like the Streaming Table , Materialized View is another core abstraction of DLT which is responsible for storing the query result in delta table.

When a Materialized View is created , we get an additional directpry called _enzyme_log. This is not present in streaming table but only within materialized view because it provides a set of optimization which manages updates and refreshes of view. It efficiently track and apply changes as it maintains metadata which makes sure that the only necessary updates or modifications are applied improving performance thus reducing unnecessary recomputation.



Materialized View with enzyme log directory

Streaming Table with No enzyme Log Directory but remember we have implemented SCD type on customer hence we get _change_data directory

How DLT allows us to enforce Data Quality which ensures valid records are only processed and stored ?

Delta Live Tables (DLT) provides built-in expectations that enable data validation and ensure only valid records are processed and stored. These expectations help maintain data integrity by enforcing specific conditions on incoming records.

DLT allows three key data quality enforcement actions:

Raise Warnings (Do Not Drop Columns): No data is dropped; instead, warnings are logged for invalid records. Helps in monitoring data quality issues without interrupting the pipeline.

```sql
CREATE LIVE TABLE customers_silver
CONSTRAINT valid_email
  EXPECT (email LIKE '%@%.%')
  ON VIOLATION WARN
AS
SELECT *
FROM LIVE.customers_bronze;
```

--Effect: If an email does not contain '@', a warning is logged, but the record

Drop Invalid Rows (ON VIOLATION DROP ROW) : Removes invalid records from the dataset while allowing valid data to be processed. Prevents bad data from entering the pipeline without stopping execution.

```
CREATE STREAMING TABLE orders_silver_cleaned (
  CONSTRAINT valid_order
    EXPECT (order_id IS NOT NULL)
    ON VIOLATION DROP ROW,

  CONSTRAINT valid_customer
    EXPECT (customer_id IS NOT NULL)
    ON VIOLATION DROP ROW
)
AS
SELECT
    order_id,
    customer_id,
    total_amount
FROM STREAM(live.orders_bronze);
```

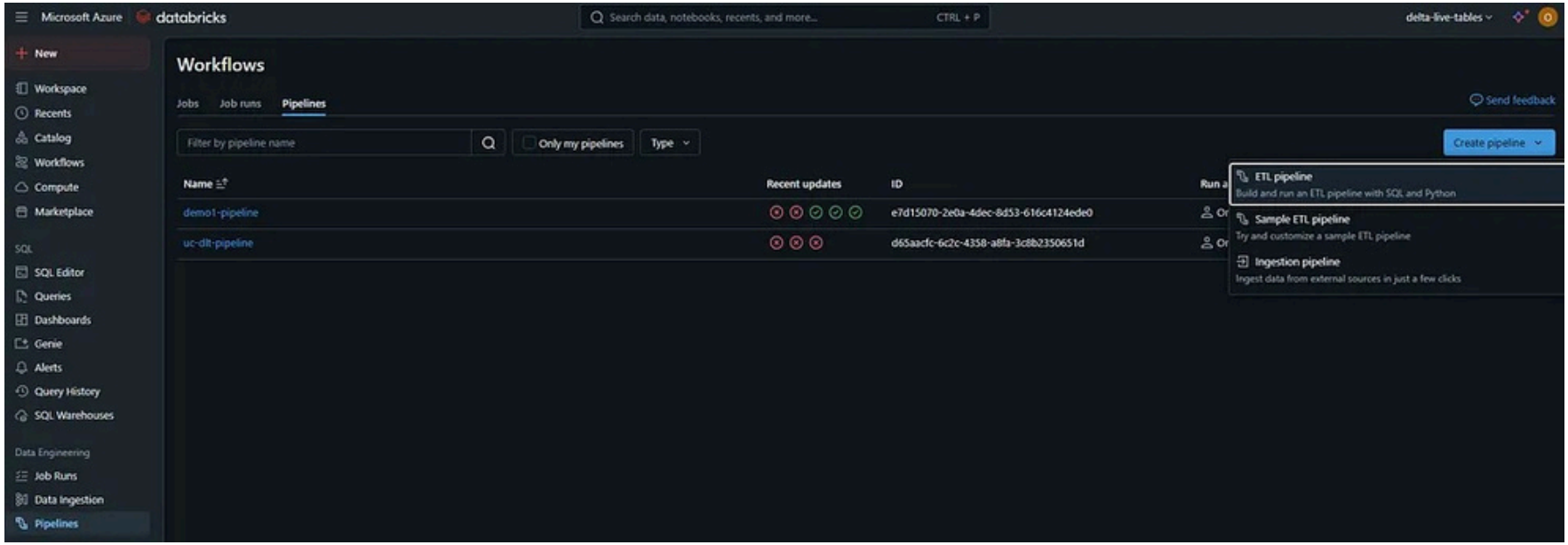
--Effect : Any record with NULL in order_id or customer_id is automatically drop

Fail the Entire Update (ON VIOLATION FAIL UPDATE) : If any record violates the expectation, the entire update is stopped. Ensures strict enforcement of data quality rules by preventing even a single invalid record from being processed.

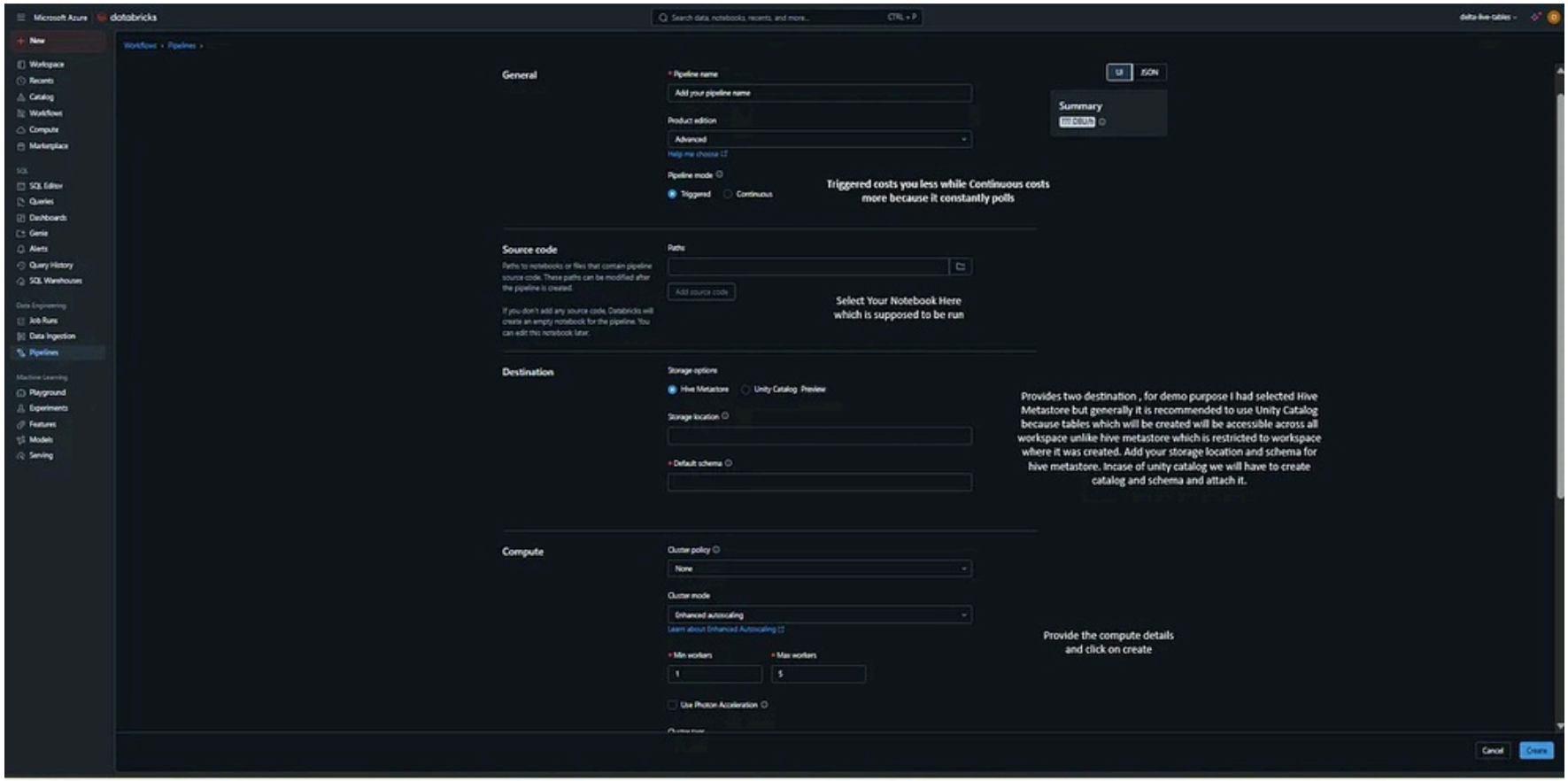
```
CREATE STREAMING TABLE transactions_silver (
  CONSTRAINT valid_transaction
    EXPECT (amount > 0)
    ON VIOLATION FAIL UPDATE
)
AS
SELECT
    transaction_id,
    amount
FROM STREAM(live.transactions_bronze);
```

--If any amount is ≤ 0, the pipeline stops completely, ensuring only valid data

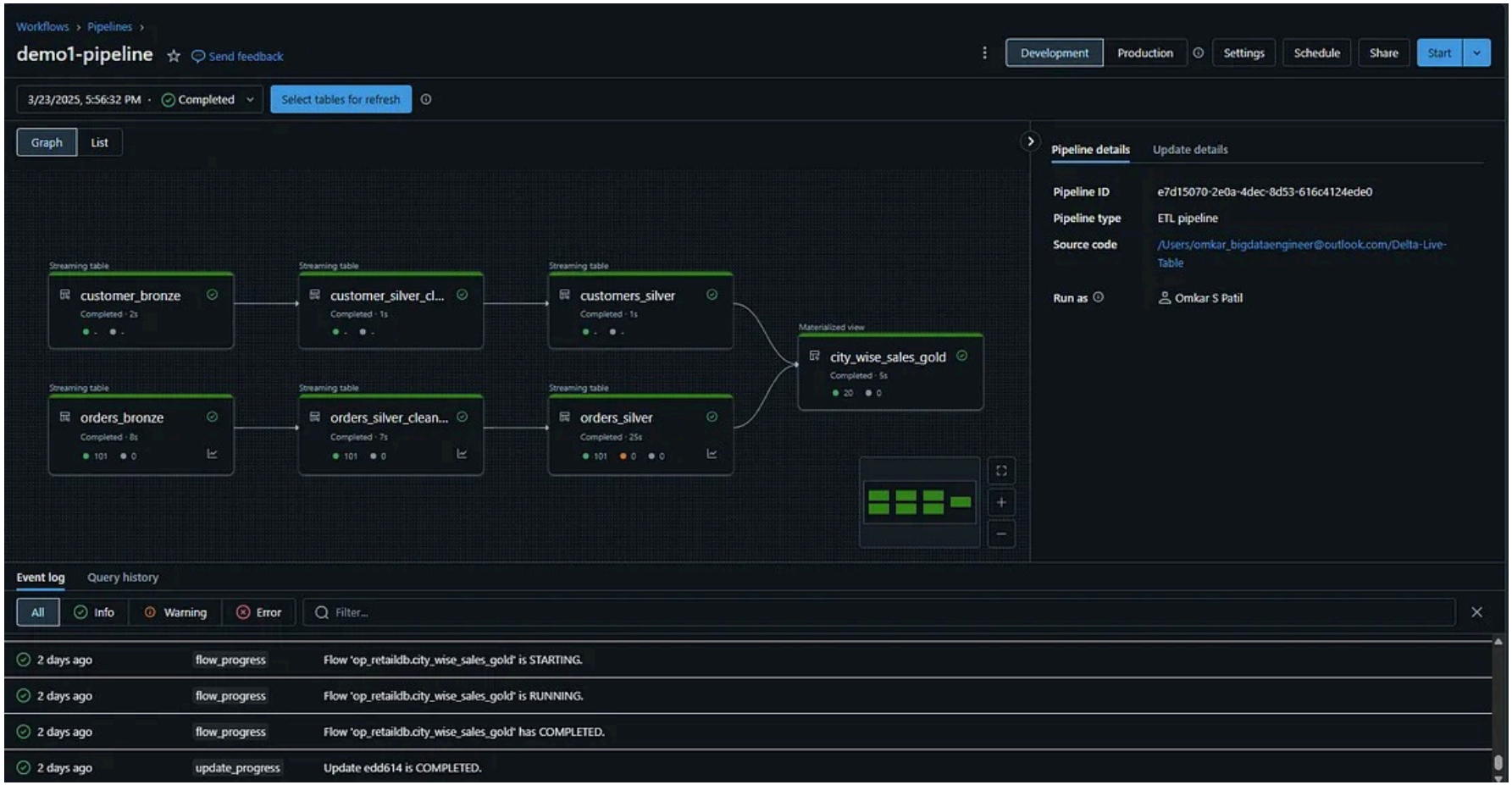
Now we have our declarative code , now it is time to run the notebook using pipeline. For this please refer the process i will be attaching in the screenshot :



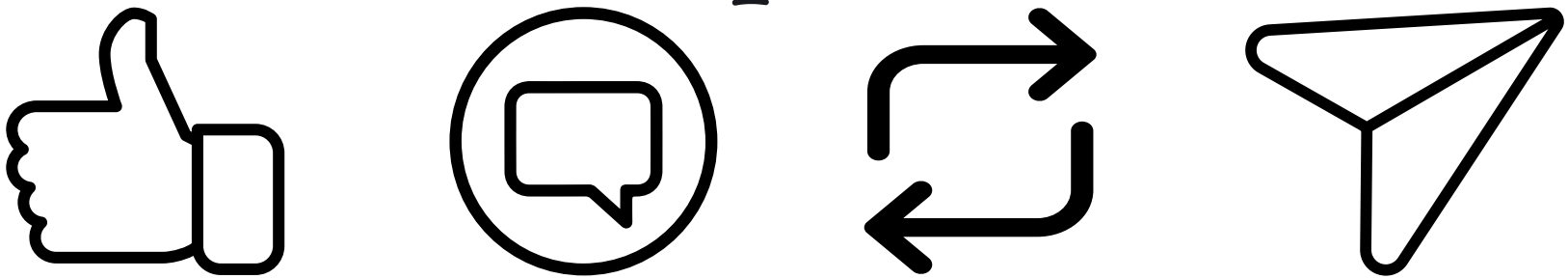
Go to pipelines > Create Pipeline > Select First Option



After creating the above pipeline , we just have to start the pipeline and it goes through certain stages and at last it shows the below screenshot which shows the data lineage and count of records inserted along with the logs


Workflows › Pipelines ›
demo1-pipeline
Send feedback
Development
Production
Settings
Schedule
Share
Start
3/23/2025, 5:56:32 PM · Completed
Select tables for refresh
Graph
List
Streaming table
customer_bronze
Completed · 2s
customer_silver_cl...
Completed · 1s
customers_silver
Completed · 1s
Materialized view
city_wise_sales_gold
Completed · 5s
orders_bronze
Completed · 8s
orders_silver_clean...
Completed · 7s
orders_silver
Completed · 25s
Pipeline details
Update details
Pipeline ID
e7d15070-2e0a-4dec-8d53-616c4124ede0
Pipeline type
ETL pipeline
Source code
/Users/omkar_bigdataengineer@outlook.com/Delta-Live-Table
Run as
Omkar S Patil
Event log
Query history
All
Info
Warning
Error
Filter...
2 days ago
flow_progress
Flow 'op_retaildb.city_wise_sales_gold' is STARTING.
2 days ago
flow_progress
Flow 'op_retaildb.city_wise_sales_gold' is RUNNING.
2 days ago
flow_progress
Flow 'op_retaildb.city_wise_sales_gold' has COMPLETED.
2 days ago
update_progress
Update edd614 is COMPLETED.

Conclusion :

DLT significantly reduces ETL complexity by automating execution, optimizing data handling, and ensuring quality enforcement. By leveraging DLT with the Medallion Architecture, data engineers can build robust, scalable, and efficient data pipelines with minimal manual intervention.

If You Found This Helpful?